

RECONVILLAGE // WORKSHOP 150 MIN

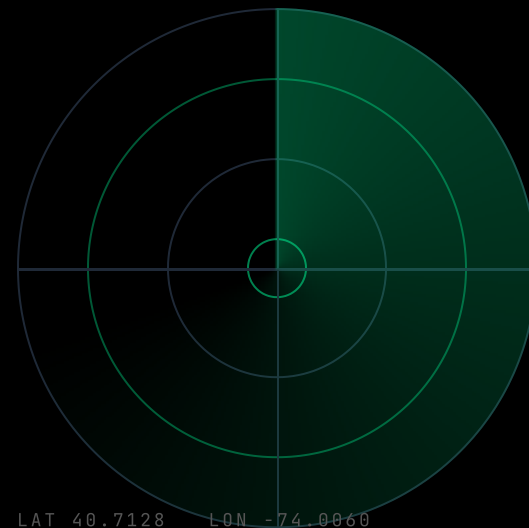
VIBE CODING AN OSINT INTELLIGENCE PLATFORM

Build a real-time geospatial intel
dashboard with AI as your engineer.

PRESENTER • HANDLE

EVENT

DATE



MISSION BRIEFING

PHASE 01

Vibe coding & AI
engineering

20 MIN

PHASE 02

Prompt
engineering

25 MIN

PHASE 03 • HANDS-ON

Build the
platform

75 MIN

PHASE 04

Lessons &
advanced
techniques

20 MIN

WHAT YOU WALK OUT WITH

01

A working OSINT globe
running on
localhost:3000

02

A repeatable AI workflow
you can reuse Monday

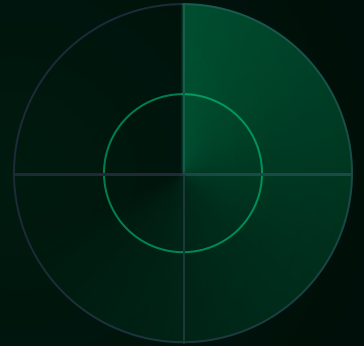
03

A recommended tool stack
that earns its cost

The app is the vehicle. The workflow is the cargo.

SECTION 01 // PART 1 • 20 MIN

VIBE CODING & AI ENGINEERING



WHAT IS VIBE CODING

Steering an AI agent to write the code while you hold intent, taste and judgement.

You describe outcomes. The agent produces diffs.
You review and redirect.

- YOU – intent, review, judgement
- AGENT – diffs, tests, iteration
- NOBODY – accepting code unread

VIBE CODING VS. AGENTIC ENGINEERING

CASUAL

"Make it work" prompting

Fast, exploratory, disposable

DISCIPLINED

Spec-driven agentic engineering

Repeatable, reviewed, production-minded

► WE ARE HERE ◀

MENTAL MODELS FOR WORKING WITH AI

- ▶ A fast, literal, forgetful junior with infinite stamina
- ▶ It optimises for your words, not your intent
- ▶ Trust, but verify
 - every claim needs evidence

THE AI-ENGINEERING VOCABULARY

CONTEXT

CONTEXT WINDOW

SYSTEM PROMPT

PROMPT

HARNESS

AGENT

AGENTIC LOOP

TOOLS

SUBAGENTS

SKILLS

MCP SERVERS

CLI TOOLS

CONTEXT & THE CONTEXT WINDOW

Everything the model can see right now.

The window is finite. It fills up, and old content gets summarised or dropped.

WINDOW OCCUPANCY

SYSTEM PROMPT



FILES



HISTORY



TOOL OUTPUT



OVERFLOW → SUMMARISED OR DROPPED

HARNESS, SYSTEM PROMPT, AGENTIC LOOP

HARNESS

Runs the model, feeds it context, executes its tool calls.

SYSTEM PROMPT

The standing instructions the harness injects every turn.

LOOP

THINK



CALL TOOL



OBSERVE



REPEAT UNTIL DONE

TOOLS, SKILLS, MCP SERVERS, SUBAGENTS

TOOLS

Actions the agent can take – read a file, run bash, search.

SKILLS

Packaged instructions and workflows, loaded on demand.

MCP SERVERS

A standard way to plug external tools and data into any agent.

SUBAGENTS

Spawned helpers that do a scoped task and report back.

THE ONE RULE

GIVE THE AGENT CLEAR, CONCISE, WELL- WRITTEN PROMPTS

Well-specified input →
good code. Vague input →
confident garbage.

PICKING THE RIGHT MODEL FOR THE TASK

CHEAP • FAST

Mechanical edits, renames, well-specified steps

TOP TIER

Architecture, specs, plans, subtle debugging

Bigger isn't always better. Cost and latency are part of the design.

COMMON MISTAKES & MISCONCEPTIONS

- "It'll just figure it out."

FIX • SPECS

- One mega-prompt for a whole app

FIX • DECOMPOSE

- No review of AI output

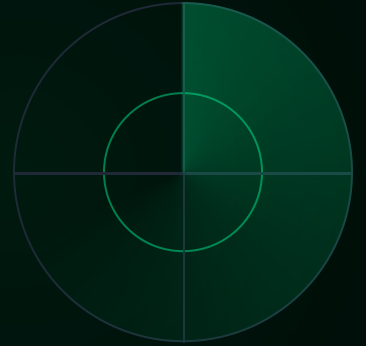
FIX • REVIEW GATES

- Treating the agent as an oracle

FIX • VERIFICATION

SECTION 02 // PART 2 • 25 MIN

PROMPT ENGINEERING



WHY THE PROMPT IS THE PRODUCT

The agent is a mirror of your instruction.

Prompting is a design activity, not a magic phrase.

IN Ambiguity



OUT Defects

ANATOMY OF A STRONG PROMPT

01 • PERSONA

02 • CORE TASK

03 • CONSTRAINTS

04 • FORMAT SPEC

05 • CHAIN OF THOUGHT (OPT.)

06 • INPUT DATA — FENCED, LAST

ATTENTION WEIGHT



Stable instructions at the top. Variable data at the bottom.

PLACEMENT MATTERS

BEFORE

```
here is our schema dump ...  
2,000 lines ...  
also make the endpoint fast  
and don't break auth
```

AFTER

```
# ROLE  
# TASK  
# CONSTRAINTS  
  - preserve auth middleware  
  - p95 < 200ms  
# INPUT  
  ```sql ... ```
```

Headers chunk attention. Lists beat prose. Bold = "this is a rule."

# TECHNIQUES THAT RELIABLY HELP

## POSITIVE FRAMING

Say what to do, not what to avoid.

## BE CONCRETE

Name the file, the function, the shape.

## DECOMPOSE

Small self-contained steps, each verifiable.

## SPECIFY THE FORMAT

Define the output exactly, up front.

# EXAMPLES & FEW-SHOT

One good example beats a paragraph of description.

Too many examples over-constrain. Use the minimum that removes ambiguity.

EXAMPLE • INPUT → OUTPUT

in: {"lat":40.71,"lon":-74.00}

out: {"id":"pt\_1","coord":[-74.00,40.71],  
"layer":"infra","color":"#00f3ff"}

# PITFALL – CONTEXT OVERLOADING

## CAUSE

Whole repos and files  
dumped into the window.

## SYMPTOMS

Forgets instructions.  
Contradicts itself. Slows  
down.

## FIX

Give only what's needed.  
Summarise. Fetch on demand.

More context is not more better.

# PITFALL — OVERFITTING & OVER-CONSTRAINING

- ▶ Tailoring to one model makes prompts brittle elsewhere
- ▶ Over-specifying removes the model's room to solve well

Precise about intent and contract. Flexible about mechanism.

# ITERATE, DON'T REGENERATE

## NUKE & RESTART

Throws away hard-won context on every hiccup.

## NUDGE & CORRECT

Fix the smallest thing. Keep the thread.

# KNOW WHEN TO TAKE THE KEYBOARD

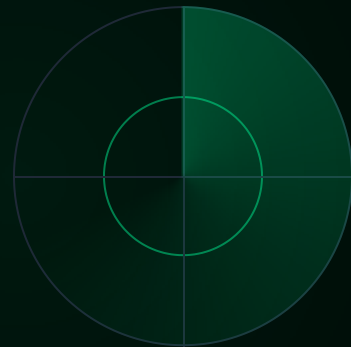
- ▶ A tricky one-liner
- ▶ A subtle merge
- ▶ A design call only  
you can make

A collaboration, not an abdication.

SECTION 03 // AGENTS • PRICING • MCPs

# THE TOOLKIT

---



# THE CODING-AGENT LANDSCAPE

## CLI AGENTS

Terminal-native,  
scriptable, close to the  
repo.

## AGENTIC IDEs

The editor is built around  
the agent.

## EDITOR EXTENSIONS

Bolt the loop onto the  
tools you already run.

Same core loop. Different surface, integrations and price.

# THE PLAYERS

## COMMAND LINE

[Claude Code](#)[Codex](#)[opencode](#)[Grok CLI](#)[Kimi CLI](#)[Gemini CLI](#)

## IDE / EDITOR

[Antigravity](#)[Cursor](#)[Windsurf](#)[Copilot agents](#)

Agent-agnostic workshop. Pick one you'll actually live in.

# REMOTE CONTROL & OBSERVABILITY

Kick off long runs

Watch progress live

Approve steps from  
anywhere

Drive and monitor an agent from your phone.

# PRICING — SUBSCRIPTION VS. API

## MONTHLY SUBSCRIPTION

Predictable. Usually cheaper for daily heavy use.

▲ RATE & USAGE CAPS

## API • PAY PER TOKEN

Flexible and scales with the work.

▲ EXPENSIVE — EASY TO OVERSPEND

Token-limited? Use the pre-made specs and plans and skip generation cost.

# MY RECOMMENDED STACK

JOB 01

## SKILLS

Methodology and current documentation.

JOB 02

## TOKEN SAVERS

Finish the build without running dry.

JOB 03

## VERIFICATION

Proof the UI actually renders.

Enough to vibe-code successfully – not a kitchen sink.

# SKILLS – SUPERPOWERS + CONTEXT7

## SUPERPOWERS

Spec-driven workflow, brainstorming, subagent execution, verification.

[github.com/obra/superpowers](https://github.com/obra/superpowers)

## CONTEXT7

Pulls current library and framework docs into context on demand.

[github.com/upstash/context7](https://github.com/upstash/context7)

# TOKEN-SAVERS — RTK + ENGRAM

## RTK

Compresses noisy build, test and git output 60-90%.

[github.com/rtk-ai/rtk](https://github.com/rtk-ai/rtk)

## ENGRAM

Persistent memory across sessions and compactions.

[github.com/Gentleman-Programming/engram](https://github.com/Gentleman-Programming/engram)

## PONYTAIL OR CAVEMAN

Optional extra context tooling. Use one, not both.

[github.com/DietrichGebert/ponytail](https://github.com/DietrichGebert/ponytail)  
[github.com/juliusbrussee/caveman](https://github.com/juliusbrussee/caveman)

# VERIFICATION — CHROME DEVTOLS MCP

Let the agent drive a real browser.

Screenshot the page. Read the console.  
Confirm it renders.

[github.com/ChromeDevTools/chrome-devtools-mcp](https://github.com/ChromeDevTools/chrome-devtools-mcp)

## CONSOLE

\$ tests: 42 passed

ERR Cesium: WebGL context lost

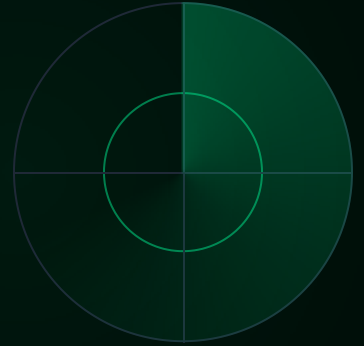
ERR 0 entities on globe

GREEN TESTS. BLANK SCREEN.

SECTION 04 // VIA SUPERPOWERS

# SPEC-DRIVEN DEVELOPMENT

---

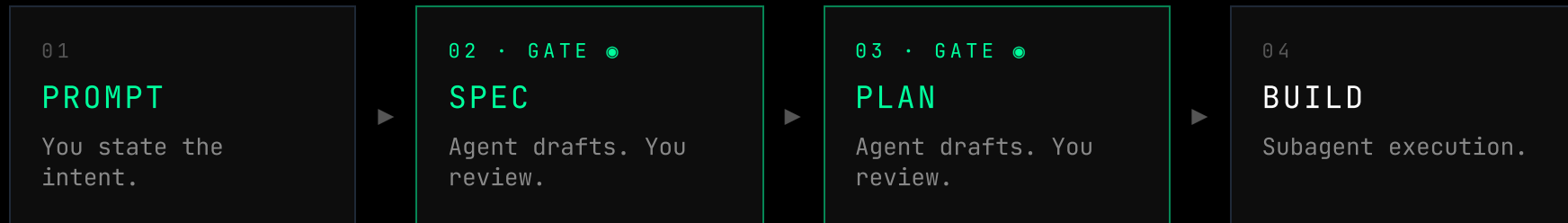


# THE PROBLEM SDD SOLVES



The agent will build the wrong thing quickly.  
Align first.

# PROMPT → SPEC → PLAN → BUILD



Two review gates before a line of app code is written.

# WHY IT WORKS

Small, verifiable steps

A shared written contract

Reviewable at every stage

Less context load per step

Catch design mistakes in the spec (minutes), not the code (hours).

# WHY SUPERPOWERS

- brainstorming
- writing-plans
- subagent-driven execution
- verification-before-completion

Turns "be disciplined" into a workflow the agent follows.

# ALTERNATIVES WORTH KNOWING

## OPENSPEC

More ceremony, more power, more overhead.

## BMAD METHOD

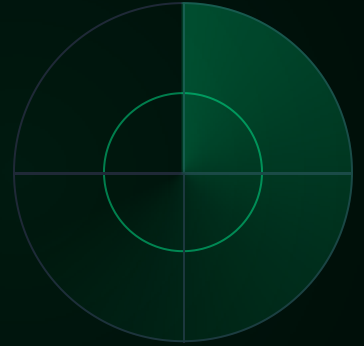
Heavier process for larger, longer programmes.

For a 75-minute build, Superpowers is the sweet spot.

SECTION 05 // WHAT WE'RE BUILDING

# THE OSINT MISSION

---



OSINT

# INSIGHT FROM PUBLICLY AVAILABLE DATA

No secrets. Just signal  
— and the work of  
fusing it.

# THE BUILD — A REAL-TIME INTELLIGENCE GLOBE

## FRONTEND

React • Redux • MUI • Cesium

## BACKEND

Node • Express • SQLite • WebSocket push

## FILTERS

CRT • Night Vision • FLIR

## LIVE LAYERS

- AIRCRAFT
- MARITIME
- SATELLITES • ISS
- EARTHQUAKES
- RADIATION
- INFRASTRUCTURE

STREAM • WS ://LOCALHOST:3000

# THE DATA — SOURCES BY DOMAIN

~55 declarative sources

## AIR

ADS-B • military flights • OpenSky  
• airports • SIGMETs • radiosondes

## MARITIME

AIS ships • world ports • NOAA  
buoys • submarine cables • AIS gap  
events

## SPACE

Celestrak satellites • ISS position  
• rocket launches • space weather

## GEOPHYSICAL

Earthquakes (EMSC/USGS) • volcanoes  
• wildfires • lightning • disasters

## ENVIRONMENT

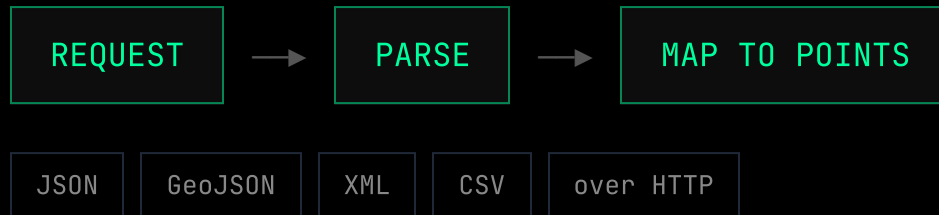
Safecast • EPA RadNet • air quality  
(OpenAQ / PurpleAir)

## GEOPOLITICAL & CYBER

ACLED • GDELT • broadcast warnings  
• peering • outages • CISA KEV

Workshop minimum: quakes • flights • ships • satellites • ISS • radiation • infra.

# DEMYSTIFYING "A SOURCE"



If you can `curl` it, you can plot it.

# SOURCES ARE DECLARATIVE YAML

TRANSPORT – URL, interval

PARSER – format

MAPPING – lat, lon, time, name

DISPLAY – icon, colour

```
sources.d/iss_position.yaml

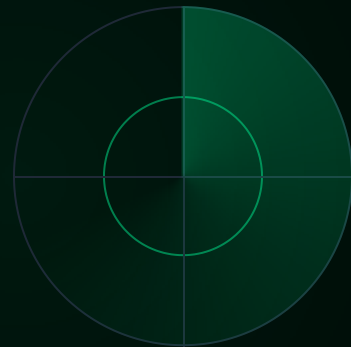
name: iss_position
transport:
 url: https://api.../iss-now.json
 interval: 5s
parser: json
mapping:
 latitude: iss_position.latitude
 longitude: iss_position.longitude
display:
 icon: satelllite
 color: "#00f3ff"
```

Add a source = write a YAML file. No engine changes.

SECTION 06 // PART 3 • 75 MIN + PART 4 • 20 MIN

# THE WORKSHOP

---



# WHAT WE'RE GOING TO DO

- ▶ Generate the frontend and the globe
- ▶ Integrate live data sources
- ▶ Build reusable components
- ▶ Debug and refactor with the agent

You drive. The agent types.

# THE CODEBASE MAP

docs/PROMPT_*.md	sequential build prompts
(6)	
docs/superpowers/specs/	one spec per step
docs/superpowers/plans/	one plan per step
skills/onboard-source/	source-onboarding skill
AGENTS.md • CLAUDE.md	agent guidance

## SIX STEPS

- 01 Setup
- 02 Ingestion engine
- 03 Backend WS API
- 04 Globe dashboard
- 05 Cinematic filters
- 06 Integration & debug

# BEFORE YOU START – SETUP CHECKLIST

## • 01 • AI MODEL

Claude • ChatGPT • Gemini  
Advanced • self-hosted (Kimi,  
Qwen, DeepSeek)

\$20 tiers work; higher tiers give  
better limits and context

## • 02 • AGENTIC TOOL

Claude Code • Codex • Cursor •  
Antigravity • Windsurf •  
Copilot Agent • Continue.dev

Installed and authenticated before  
we start

## • 03 • DEV ENVIRONMENT

Git • Node.js LTS • VS Code •  
internet • can clone from  
GitHub

Repo cloned • `make install` runs  
clean

• Superpowers • RTK • Engram • Context7 • DevTools MCP

• Ponytail or Caveman – never both

Don't start Part 3 until all green.

# CLONE THE REPO

## TERMINAL

```
$ git clone git@github.com:Alevsk/vibe-coding-osint-platform.git
$ cd vibe-coding-osint-platform
$ make install
```

BROWSE <https://github.com/Alevsk/vibe-coding-osint-platform>

No SSH keys? Use the HTTPS URL from the repo page.

# TWO PATHS THROUGH THE BUILD

## PATH A • FULL SPEC-DRIVEN

Run `PROMPT_N.md` → agent writes spec + plan →  
you review both → subagent execution.

Teaches the method.

## PATH B • TOKEN-SAVER

Use the provided specs and plans → "use a  
subagent-driven approach to execute plan 1"  
→ review → plans 2-6.

Saves tokens on limited accounts.

Same destination.

# VALIDATE AT EVERY MILESTONE

Watch the app live at  
<http://localhost:3000>

After each plan, exercise it – then fix or  
continue.

Don't batch six steps blind.

AGENT PROMPT

\$ fix these bugs: ...

\$ continue to the next plan

# EXTEND IT — ONBOARD NEW SOURCES

GIVE IT A URL



FETCH & ANALYSE FORMAT



WRITE `sources.d/*.yaml`

```
skills/onboard-source/SKILL.md
```

Bolt on any public feed and watch it hit the globe.

# WHAT AI DOES WELL • WHERE IT STRUGGLES

## DOES WELL

Boilerplate

Wiring things together

Breadth across libraries

Tireless iteration

Debugging with evidence

## STRUGGLES

Ambiguous intent

Novel architecture

Silent wrong-confidence

Long-horizon memory

Keep humans on judgement and verification.

# TECHNICAL DEBT, COST, WORKFLOW

## DEBT

Review diffs. Keep steps small. Refactor deliberately.

## COST

Subscription + RTK/Engram. Reuse specs. Cheap models for mechanical steps.

THE LOOP SPEC ► PLAN ► BUILD ► VERIFY ► REPEAT

## RECAP

# VIBE CODING IS ENGINEERING WHEN YOU ADD SPECS, PLANS, REVIEW

You built a live OSINT  
globe by directing AI.

# RESOURCES & LINKS

Superpowers – [github.com/obra/superpowers](https://github.com/obra/superpowers)

Context7 – [github.com/upstash/context7](https://github.com/upstash/context7)

RTK – [github.com/rtk-ai/rtk](https://github.com/rtk-ai/rtk)

Engram – [github.com/Gentleman-Programming/engram](https://github.com/Gentleman-Programming/engram)

Chrome DevTools MCP – [github.com/ChromeDevTools/chrome-devtools-mcp](https://github.com/ChromeDevTools/chrome-devtools-mcp)

Ponytail / Caveman – [github.com/DietrichGebert/ponytail](https://github.com/DietrichGebert/ponytail) ·  
[github.com/juliusbrussee/caveman](https://github.com/juliusbrussee/caveman)



Repo · sources catalogue

# // END OF BRIEFING

QUESTIONS — THEN GO BUILD